

Trabalho Prático 2 — Phaser 3 (Época de Recurso)

Tecnologias Multimédia 2025/2026

0. Pontos a ter em atenção

Leiam **antes** de começar. Estes pontos são vinculativos.

Item	Resumo
Motor	Phaser 3 (2D, corre no browser).
Tema	Livre — qualquer género 2D.
Localização do projeto	Na raiz do repositório (não dentro de subpasta).
Modalidade	Individual ou em par (2 alunos) , à escolha dos estudantes.
Suporte multilíngue	Obrigatório: ≥ 2 línguas (ex.: PT + EN).
Tamanho de ficheiros / carregamento	Avaliado — sprites proporcionais, áudio comprimido, sem assets sobredimensionados.
Som	Pelo menos 1 som integrado no jogo.
Publicação online	Proibida. Apenas GitHub Pages como bónus opcional.
Entrega	Tag <code>1.0</code> na versão entregue, README completo, demonstração presencial obrigatória.

1. Objetivos

- Desenvolver um **jogo 2D no browser** usando **Phaser 3**, demonstrando compreensão das mecânicas de jogo e da programação em JavaScript.
- Aplicar, no mínimo, os conceitos trabalhados nas aulas práticas:
 - Cenas (`preload` , `create` , `update`)
 - Sprites e imagens
 - Física Arcade (grupos, velocidades, colisões/overlap)
 - Input (teclado e/ou rato)
 - Estado de jogo (pontuação, vidas/objetivo, Game Over/Vitória)
- Integrar **recursos multimédia** (imagem e som) de forma adequada ao tema escolhido.

2. Descrição do jogo

A **ideia do jogo é livre**, desde que seja desenvolvida em **Phaser 3** e demonstre domínio das mecânicas trabalhadas nas aulas. Exemplos de géneros possíveis (não exaustivo):

- *Shooter, avoidance* (top-down ou lateral)
- *Platformer* simples
- *Puzzle / match-3*
- *Endless runner*
- *Top-down adventure* curto
- *Tower defense* simples

Nota sobre 2D/3D: o Phaser é um motor **2D**. Pode usar-se perspectiva top-down, lateral, isométrica ou pseudo-3D (escala/parallax), mas não é esperado nem exigido 3D real.

Independentemente do género, o jogo deve incluir, no mínimo:

- **Jogador** controlado pelo utilizador (teclado e/ou rato), com movimento ou interação clara.
- **Elementos de jogo** que reagem ao tempo ou ao input (inimigos, obstáculos, alvos, peças, recursos...).
- **Colisões/interações** detetadas via física Arcade (`overlap` / `collider`) ou por outra via justificada.
- **Estado de jogo visível:** pontuação, progresso, vidas, tempo ou outro indicador adequado ao tema.
- **Condições de fim:** *Game Over* e/ou *Vitória*, com possibilidade de **reinício** (ex.: tecla R, botão).
- **Pelo menos um som integrado** (efeito ou música) — requisito da componente multimédia.
- **Suporte a pelo menos 2 línguas** (ex.: PT + EN) com seletor acessível (menu ou opção em-jogo). Toda a UI textual deve ser traduzida; estrutura de tradução coerente (ex.: ficheiro JSON por língua) — não strings duplicadas no código.
- **Tamanho de ficheiros e tempos de carregamento controlados:** sprites com resolução proporcional ao uso (sem PNG 4096 para sprite 32px), áudio comprimido (OGG/MP3) em vez de WAV bruto onde aplicável, total de assets razoável (idealmente <10 MB). Sem assets não usados no repositório.

Funcionalidades adicionais (múltiplas cenas, *power-ups*, partículas, banda sonora completa, línguas extra, etc.) são valorizadas como **pontos extra** — ver secção 6.

3. Requisitos técnicos

- **Engine:** Phaser 3 (versão estável, ex.: 3.80 ou superior). Pode ser incluída por **CDN** ou via npm.

- **Linguagem:** JavaScript (ES6+). TypeScript é aceite se o aluno justificar a escolha.
- **Física:** uso do sistema de física Arcade quando aplicável ao género (grupos, `setVelocity`, `overlap/collider`).
- **Estrutura do projeto:** organização **livre mas legível** — a decisão final é do developer. Como referência, segue uma estrutura padrão para projetos Phaser 3 (não obrigatória):

```

meu-jogo/
├── index.html           # carrega Phaser e o script principal
├── package.json         # se usar npm (opcional)
├── .gitignore
├── README.md
├── src/ (ou js/)
│   ├── main.js         # config do jogo, registo de cenas
│   └── scenes/
│       ├── BootScene.js
│       ├── PreloadScene.js
│       ├── MenuScene.js
│       ├── GameScene.js
│       └── GameOverScene.js
└── assets/
    ├── images/         # sprites, fundos
    ├── spritesheets/   # animações
    ├── audio/          # efeitos e música
    └── fonts/          # bitmap fonts (opcional)

```

Variantes aceitáveis: tudo num só `game.js` para projetos pequenos; `public/ + src/` se usar bundler (Vite/webpack); separar por *feature* em vez de por tipo. **O critério é coerência e legibilidade**, não conformidade com este esqueleto.

- **Execução:** o jogo deve correr num navegador moderno (Chrome, Firefox, Edge), servido por um **servidor HTTP local** (Live Server, `npx serve`, `npm start`...). Não é exigido que funcione apenas via `file://`.

4. Entrega

4.1 Repositório

- O projeto deve estar num **repositório GitHub**.
- O repositório deve ser **público** ou o docente deve ter **permissão de visualização**.
- **O projeto Phaser deve estar na raiz do repositório** (não dentro de uma subpasta). Documentação, planeamento e demais material podem coexistir na raiz, devidamente organizados.
- Utilizar um **.gitignore** adequado para projetos Node/Phaser. Não versionar `node_modules/`.

4.2 Ficheiro de entrega

- **Prazo de entrega e apresentação:** até **17 de julho de 2026, 23:59**. A entrega no Moodle e a apresentação ao docente devem ocorrer até essa data; o dia concreto da apresentação é **combinado com o docente**.
- **Onde entregar:** ficheiro de texto submetido no **Moodle**.
- **Nome do ficheiro:** `numeroaluno1_numeroaluno2.txt` (em par) ou `numeroaluno.txt` (individual).
- **Conteúdo do ficheiro:**
 - **URL do repositório GitHub;**
 - **Commit hash** da versão entregue;
 - **Nome e número** de cada elemento.
- **Tag no repositório:** a versão entregue deve estar identificada com a tag `1.0` (criar a tag e fazer *push* para o GitHub).

Modalidade: o trabalho pode ser realizado **individualmente ou em grupo de 2 alunos**, à escolha dos estudantes.

4.3 README obrigatório

No repositório deve existir um **README.md** com, no mínimo:

- **Nome e número** de cada elemento do grupo (ou do aluno, se individual).
- **Indicação clara** de que se trata da entrega da **época de recurso**.
- **Versão de Phaser** usada (ex.: 3.80) e como foi incluída (CDN ou npm).
- **Descrição do jogo:** género, objetivo, regras e funcionalidades implementadas.
- **Jogabilidade / controlos:** teclas e ações.
- **Como executar:** instruções claras (ex.: `npm install` e `npm start`, Live Server, `npx serve`).
- **Aspectos multimédia:** formatos usados (imagens, sons, *spritesheets*), origem/criação dos *assets*, resolução/tamanho e breve justificação.
- (Opcional) Capturas de ecrã ou GIF, *roadmap*, lacunas conhecidas. Se ativaram **GitHub Pages** (bónus opcional — ver secção 6), podem incluir o link.

4.4 Demonstração

- **Demonstração obrigatória**, na data combinada com o docente, até ao **prazo de 17 de julho de 2026, 23:59**.
- **Duração:** até **5 minutos** de apresentação + **10 minutos** de perguntas.
- A demonstração é condição de aprovação.

5. Condições de avaliação

Critério	Descrição
Funcionalidade	Mecânicas implementadas e funcionais para o género escolhido: input, interações/colisões, estado

Critério	Descrição
	de jogo (score/vidas/progresso), Game Over/Vitória e reinício.
Código e estrutura	Organização (cenas, funções/módulos auxiliares), legibilidade, sem duplicação desnecessária.
Uso de Phaser	Uso correto e adequado de sprites, grupos, física, input, <i>overlap/collider</i> , cenas, <i>tweens</i> /animações onde aplicável.
Multimédia (qualidade) e relatório	README com secção <i>Aspectos multimédia</i> (formatos, origem, justificação). Pelo menos um som integrado no jogo. Coerência visual/sonora.
Tamanho de ficheiros e tempos de carregamento	Resolução de sprites proporcional ao uso, áudio comprimido, total de assets razoável, sem assets não usados.
Suporte multilíngue (i18n)	≥ 2 línguas com seletor acessível, UI 100% traduzida, estrutura de tradução coerente (não strings hardcoded espalhadas pelo código).
Demonstração	Apresentação clara, defesa do código, compreensão individual de cada elemento.
Repositório e entrega	Git com histórico coerente, <code>.gitignore</code> adequado, tag <code>1.0</code> na versão entregue, README completo. Projeto corre ao abrir.

6. Pontos extra (valorizados)

- **Entrega e apresentação antecipadas:** pontuação extra para entregas e apresentações antes da data limite (a combinar com o docente).
- **Criatividade:** originalidade do conceito, narrativa, arte ou mecânicas.
- Múltiplas cenas (menu, jogo, *game over*, vitória).
- Diversidade de inimigos/obstáculos/níveis ou padrões de movimento.
- *Power-ups*, sistemas de progressão ou economia interna.
- Aumento progressivo de dificuldade.
- Animações com *spritesheets*, partículas, *tweens* e *cameras* (shake, fade, follow).
- Banda sonora completa: música de fundo + efeitos distintos por evento.
- Interface (UI) cuidada com pontuação, vidas e instruções no jogo.
- 3+ línguas suportadas (vai além do mínimo de 2).

Nota sobre publicação online: o jogo **não deve ser publicado** — fica apenas no repositório GitHub e corre localmente (no computador do grupo ou do docente para avaliação). Como bónus *menor*, podem opcionalmente ativar **GitHub Pages** (gratuito para repositórios públicos) e incluir o link no README. Não usem outros serviços.

Nota sobre frameworks: **React**, **Vue** ou outras frameworks/bundlers (Vite, webpack, Parcel, Svelte, etc.) são **completamente aceitáveis** — desde que o jogo continue a usar **Phaser 3** como motor e que o projeto corra com instruções claras no README. A escolha é vossa.

7. Referências

- Phaser 3: phaser.io · [documentação](#) · [exemplos](#)